МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ СИСТЕМ

Лабораторна робота №3

Практикум із розроблення через тестування

Цикл Red – Green – Refactor

Варіант №12

Розрахунок вартості страховки

Виконала:
Студентка 4 курсу
Денної форми навчання
Групи ІПЗ-1
Граб Емілія Володимирівна

Ужгород – 2026

1 Мета роботи та індивідуальний варіант

Мета роботи – опанувати розроблення через тестування, реалізувати модулі з нуля за циклом Red – Green – Refactor, фіксувати кожний етап окремим комітом та виконувати рефакторинг під захистом тестів.

Виконано показовий тарифікатор паркування та індивідуальний варіант №12 – калькулятор вартості страховки. Підсумковий набір містить 254 успішні тести; покриття гілок страхування становить 100%. Реальна історія містить 16 завершених циклів TDD, по вісім для кожного модуля.

Основне джерело – повний файл «Лабораторна 3_Практикум TDD.docx». Його витяг збережено у report/lab3/evidence/methodology.txt, а контрольну суму – у methodology-sha256.txt. Числові припущення для страхування нижче відокремлено від прямих вимог методички.

До початку змін робоче дерево було чистим; вихідна гілка – feature/lab2-vitest-installment, коміт 3219dc2. Створено feature/lab3-tdd-insurance. Початкові 145 тестів лабораторних №1 і №2 проходили; їхні вихідні файли та тести не змінено. Середовище: TypeScript 5.8, Vitest 5.0.3, покриття V8, ESLint і Prettier.

2 Розбиття специфікації на вимоги

2.1 Явні вимоги та прийняті припущення

Методичка задає базовий тариф залежно від віку авто, надбавку за стажу водія менше трьох років та знижку за безаварійність до 30%. Ціни, категорії віку, величину надбавки й спосіб накопичення знижки не задано. Наведені далі тарифи є навчальними припущеннями автора.

Таблиця 1 – Політика страхового калькулятора

Параметр	Правило та походження
Базова ціна	Припущення: 0–3 повні роки – 3000 грн; 4–10 – 4000 грн; 11+ – 5000 грн.

Параметр	Правило та походження
Стаж	Вимога: надбавка за стажу <3 років. Припущення: надбавка 25%; дробові роки дозволено.
Безаварійність	Вимога: знижка максимум 30%. Припущення: передається вже встановлене право на знижку у відсотках; нарахування права поза модулем.
Знижка	Необов’язковий невід’ємний скінченний відсоток, типово 0; дробові та більші за 30 значення дозволено, діє обмеження 30.
Порядок	Базова ціна → надбавка → знижка від збільшеної суми → округлення до копійок, половина вгору.
Валідація	Вік – невід’ємне безпечне ціле число; стаж і знижка – скінченні невід’ємні числа. Помилки RangeError з кодом відповідного поля.

Контракт: calcInsuranceCost({ vehicleAge, driverExperience, accidentFreeDiscountPercent? }): number. Усі бізнесові ціни, межі та ставки мають іменовані константи. Значення 10 і 25% не приписуються методичці; безпосередньо заданими є лише межа стажу 3 роки та максимум знижки 30%.

Таблиця 2 – Вимоги варіанта №12 у порядку реалізації

Цикл	Нова вимога	Фактичний Red та Green
1	Валідація віку, стажу й відсотка	Відсутній модуль; після Green – 202 тести.
2	Базова ціна першої категорії 3000 грн	Замість 3000 отримано 0; Green – 206.
3	Категорії 4–10 та 11+ років	Отримано 3000 замість 4000/5000; Green – 213.

Цикл	Нова вимога	Фактичний Red та Green
4	Надбавка 25% за стажу <3 років	Отримано 3000 замість 3750; Green – 221.
5	Знижка за безаварійність після надбавки	Немає знижки, зокрема 3000 замість 2700; Green – 227.
6	Максимум знижки 30%	31% зменшували суму до 2070 замість 2100; Green – 235.
7	Фінальне округлення та взаємодія правил	Отримано 2629,6353 замість 2629,64; Green – 247.
8	Точність десяткової половини копійки	2999,77 замість 2999,78 при 0,0075%; Green – 253.

Кожний рядок таблиці відповідає окремим Red, Green і Refactor. Поведінку наступної категорії або знижки не додавали до відповідного Red. У першому Green страхування коректні вхідні дані ще давали 0, у другому – одну ціну для всіх вікових категорій; наступні тести змусили розширити ці реалізації.

2.2 Показовий тарифікатор паркування

Таблиця 3 – Послідовність вимог паркування

Цикл	Вимога	Повний набір після Green
1	INVALID_DURATION для від’ємної, дробової та нескінченної тривалості	151
2	0–15 хв безкоштовно	155
3	16–60 хв – 30 грн	158
4	Кожна наступна розпочата година – 20 грн	163
5	Обмеження базової плати до 200 грн	169

Цикл	Вимога	Повний набір після Green
6	Нічний множник 0,5 після обмеження	175
7	Вихідний множник 0,8 після нічного	181
8	Округлення до двох знаків	185

Межові значення: 14/15/16, 59/60/61, 119/120/121 та 539/540/541 хв. 61 хв коштує 50 грн, 121 хв – 70 грн, 540 хв – 190 грн, 541 хв – 200 грн. Для 600 хв з обома прапорцями: 200 × 0,5 × 0,8 = 80 грн. Обмеження застосовується до знижок; для 600 хв спочатку поділити 210 навпіл означало б помилкові 105 грн замість 100.

Приклади 12 і 28 грн із восьмого циклу методички вже проходили після сьомого: усі задані тарифи дають цілі результати. Тому не стверджується, що вони були Red. Реальний Red восьмого циклу перевіряв нову roundParkingAmount на 12,344/12,345/12,346; виклик відсутньої функції впав, після мінімальної реалізації – пройшов. Публічний калькулятор застосовує її до остаточної суми.

2.3 Межі та незалежні очікувані значення

Таблиця 4 – Перевірка числових порогів страхування

Межа	Вхідні значення	Очікування
Вік 0	−1, 0, 1; дробове 0,5	Відмова, 3000, 3000; відмова
Вік 3	2, 3, 4	3000, 3000, 4000
Вік 10	9, 10, 11	4000, 4000, 5000
Безпечний вік	MAX_SAFE_INTEGER −1; межа; +1	5000; 5000; відмова
Стаж 0	−0,01; 0; 0,01	Відмова; 3750; 3750 для авто віком 0

Межа	Вхідні значення	Очікування
Стаж 3	2; 3; 4; також 2,99/3,01	3750; 3000; 3000; 3750/3000
Знижка 0	−0,01; 0; 0,01%	Відмова; 3000; 2999,70
Знижка 30	29/30/31%; також 29,99/30,01%	2130/2100/2100; 2100,30/2100
Половина копійки	0,0074999/0,0075/0,0075001%	2999,78/2999,78/2999,77

Очікування у it.each записані явними числами. Наприклад, 5000 + 1250 надбавки − 1875 знижки = 4375 грн. Для знижки 0,0345%: 3000 − 1,035 = 2998,965, після округлення – 2998,97 грн. Такі перевірки не повторюють програмну формулу обчислення очікуваного значення.

3 Історія комітів і реальний процес TDD

Знімок команди git log --oneline отримано через Playwright із переглядача незміненого виводу. Команда показує нові коміти першими; хронологічний порядок від старих до нових збережено окремо командою git log --reverse --oneline 3219dc2..HEAD. Нижче наведено хеші всіх 48 етапів; повний знімок git-history.png і журнали містяться у репозиторії.

У верхній частині історії видно завершення восьмого страхового циклу та попередні цикли. Повний журнал містить і всі вісім циклів паркування.



Рисунок 1 – Фактичний вивід git log --oneline у браузері

Таблиця 5 – Окремі коміти кожного циклу П – паркування С – страхування

Цикл	Red	Green	Refactor
П01	f114db2	20b15d1	08f887f
П02	f9e5334	a8d4761	3dd6f93
П03	a2edd78	ed71d00	409ac8c
П04	f610f1f	132132c	bd1636b
П05	7be2a89	5b45d54	0692c42
П06	a9f4b87	2b91254	9dcb104
П07	84faebc	b058950	c146b6d
П08	4d952f3	e676de8	684df52

Цикл	Red	Green	Refactor
C01	3960548	558d84e	54daef5
C02	04f36be	0365113	ffbcb76
C03	a56f0ca	5434792	00fc60b
C04	14ca255	154bf30	f6587b0
C05	010d357	2efe14c	9540017
C06	bdb20a6	ca70449	898352b
C07	b84689e	cdb501e	e779156
C08	665dd92	daecb80	dcd4409

Автоматичний аудит перевірив батьківський коміт, порядок етапів, незмінність тестів у Green, відсутність змін продуктового файлу в Red і збіг знімків коду з вмістом відповідного коміту. Усі 16 Red мають exit 1, усі 16 Green та 16 Refactor – exit 0. Перші Red обох модулів – помилки імпорту, як дозволяє методичка.

Red страхування, цикл 2: чотири сценарії віку 0–3 роки очікували 3000 грн, проте попередня мінімальна реалізація повертала 0. Тест падав саме через відсутність базової ціни.

insurance-02 RED

Execution-log viewer — original captured process output, not a terminal emulator.

$ npm test -- tests/insurance.test.ts | exit 1 | 2026-10-26T20:57:53.451Z | stdout followed by stderr

```
> testing@1.0.0 test
> vitest run tests/insurance.test.ts


 RUN  v5.0.3 /Users/mila/WebstormProjects/unik/testing

 ❯ tests/insurance.test.ts (21 tests | 4 failed) 4ms
   × base cost for a young vehicle aged 0 2ms
   × base cost for a young vehicle aged 1 0ms
   × base cost for a young vehicle aged 2 0ms
   × base cost for a young vehicle aged 3 0ms

 Test Files  1 failed (1)
      Tests  4 failed | 17 passed (21)
   Start at  22:57:53
   Duration  67ms (transform 45%, tests 24%, import 22%, worker 10%)


⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯ Failed Tests 4 ⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯

 FAIL  tests/insurance.test.ts > base cost for a young vehicle aged 0
 FAIL  tests/insurance.test.ts > base cost for a young vehicle aged 1
 FAIL  tests/insurance.test.ts > base cost for a young vehicle aged 2
 FAIL  tests/insurance.test.ts > base cost for a young vehicle aged 3
AssertionError: expected +0 to be 3000 // Object.is equality

- Expected
+ Received

- 3000
+ 0

 ❯ tests/insurance.test.ts:5:66
      3|
      4| it.each([0, 1, 2, 3])('base cost for a young vehicle aged %i', (vehicl…
      5|   expect(calcInsuranceCost({ vehicleAge, driverExperience: 3 })).toBe(…
       |                                                                  ^
      6| });
      7|

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯
```

Green того самого циклу: мінімальне повернення 3000 грн задовольнило нові тести. Категорії старших автомобілів ще не реалізовувалися. Повний набір із 206 тестів залишився успішним.

insurance-02 GREEN

Execution-log viewer — original captured process output, not a terminal emulator.

$ npm test | exit 0 | 2026-10-26T20:57:56.039Z | stdout followed by stderr

```
> testing@1.0.0 test
> vitest run


 RUN  v5.0.3 /Users/mila/WebstormProjects/unik/testing


 Test Files  6 passed (6)
      Tests  206 passed (206)
   Start at  22:57:56
   Duration  90ms (transform 56%, import 23%, tests 11%, worker 9%)
```

Рисунок 3 – Ті самі тести після мінімальної реалізації

Після виділення calcBasePremium та іменованих меж у третьому циклі пройшли всі 213 тестів. Змінено структуру функцій, а ціни та межі, зафіксовані попередніми тестами, збережено.

insurance-03 REFACTOR

Execution-log viewer — original captured process output, not a terminal emulator.

$ npm test | exit 0 | 2026-10-26T20:58:20.984Z | stdout followed by stderr

```
> testing@1.0.0 test
> vitest run


 RUN  v5.0.3 /Users/mila/WebstormProjects/unik/testing


 Test Files  6 passed (6)
      Tests  213 passed (213)
   Start at  22:58:21
   Duration  86ms (transform 50%, import 26%, tests 14%, worker 9%)
```

Рисунок 4 – Успішний набір після структурного рефакторингу

Додатковий восьмий цикл виник із перевірки точності: стандартна арифметика Number занижувала дві десяткові половини копійки. Red дав 2 невдалі тести з 68, Green – усі 253 успішні. Точний десятковий запис відсотка перетворено на цілочислову частку; округлення виконано один раз після всіх тарифних правил.

Під час остаточного аудиту додано ще одну перевірку MAX_SAFE_INTEGER − 1, щоб мати всі три сусідні точки межі безпечного цілого. Вона одразу пройшла без змін продуктового коду. Це додаткова перевірка наявного контракту, а не новий Red або TDD-цикл; фінальний набір зріс із 253 до 254 тестів.

Технічна примітка до доказів: у parking-03 збережено додатковий незакомічений діагностичний запуск. Перевірка шаблону журналу спочатку не врахувала позначення +0 у Vitest, тому запис коміту було відхилено. Без зміни тесту чи реалізації запуск повторено з виправленим шаблоном. Це не окремий TDD-цикл і не прихований збій продуктового коду.

4 Кінцеві лістинги тестів і продуктового коду

Лістинги наведено повністю у фінальному стані. Паркування розділяє перевірку тривалості, базову ціну, застосування прапорців і округлення. Страхування відокремлює валідацію від вибору базової ціни, надбавки та точного фінального перерахунку в копійки.

Лістинг 1 src/parking.ts

```
const INVALID_DURATION = 'INVALID_DURATION';
const FREE_MINUTES = 15;
const FIRST_HOUR_FEE = 30;
const MINUTES_PER_HOUR = 60;
const EXTRA_HOUR_FEE = 20;
const DAILY_CAP = 200;
const NIGHT_RATE = 0.5;
const WEEKEND_RATE = 0.8;
const KOPECKS_PER_UAH = 100;

export interface ParkingOptions {
  night?: boolean;
  weekend?: boolean;
```

```
}

export function roundParkingAmount(amount: number): number {
  return Math.round(amount * KOPECKS_PER_UAH) / KOPECKS_PER_UAH;
}

function assertValidDuration(minutes: number): void {
  if (!Number.isInteger(minutes) || minutes < 0) {
    throw new Error(INVALID_DURATION);
  }
}

function calcBaseFee(minutes: number): number {
  if (minutes <= FREE_MINUTES) return 0;
  if (minutes <= MINUTES_PER_HOUR) return FIRST_HOUR_FEE;
  const extraHours = Math.ceil((minutes - MINUTES_PER_HOUR) /
MINUTES_PER_HOUR);
  return Math.min(FIRST_HOUR_FEE + extraHours * EXTRA_HOUR_FEE,
DAILY_CAP);
}

function applyRates(fee: number, options: ParkingOptions): number {
  if (options.night) fee *= NIGHT_RATE;
  if (options.weekend) fee *= WEEKEND_RATE;
  return fee;
}

export function calcParkingFee(
  minutes: number,
  options: ParkingOptions = {},
```

```
): number {
  assertValidDuration(minutes);
  return roundParkingAmount(applyRates(calcBaseFee(minutes), options));
}
```

Лістинг 2 tests/parking.test.ts

```
import { expect, it } from 'vitest';
import { calcParkingFee, roundParkingAmount } from '../src/parking';

it.each([
  [12.344, 12.34],
  [12.345, 12.35],
  [12.346, 12.35],
  [28, 28],
])('rounds the final amount %s to %s', (amount, expected) => {
  expect(roundParkingAmount(amount)).toBe(expected);
});

it.each([-1, -60, 10.5, NaN, Infinity, -Infinity])(
  'rejects invalid duration %s',
  (minutes) => {
    expect(() => calcParkingFee(minutes)).toThrow('INVALID_DURATION');
  },
);

it.each([0, 1, 14, 15])('first 15 minutes are free: %i', (minutes) => {
  expect(calcParkingFee(minutes)).toBe(0);
});

it.each([
```

```
  { minutes: 600, options: { weekend: true }, expected: 160 },
  { minutes: 600, options: { night: true, weekend: true }, expected: 80 },
  { minutes: 45, options: { night: true, weekend: true }, expected: 12 },
  { minutes: 121, options: { night: true, weekend: true }, expected: 28 },
  { minutes: 0, options: { weekend: true }, expected: 0 },
  { minutes: 16, options: { night: false, weekend: false }, expected: 30 },
])(
  'weekend $minutes $options costs $expected',
  ({ minutes, options, expected }) => {
    expect(calcParkingFee(minutes, options)).toBe(expected);
  },
);

it.each([
  [0, 0],
  [15, 0],
  [16, 15],
  [60, 15],
  [121, 35],
  [600, 100],
])('night rate %i costs %i', (minutes, expected) => {
  expect(calcParkingFee(minutes, { night: true })).toBe(expected);
});

it.each([
  [539, 190],
  [540, 190],
  [541, 200],
  [600, 200],
  [1440, 200],
```

```
  [1441, 200],
])('daily cap %i costs %i', (minutes, expected) => {
  expect(calcParkingFee(minutes)).toBe(expected);
});

it.each([
  [61, 50],
  [119, 50],
  [120, 50],
  [121, 70],
  [180, 70],
])('started extra hours %i cost %i', (minutes, expected) => {
  expect(calcParkingFee(minutes)).toBe(expected);
});

it.each([
  [16, 30],
  [59, 30],
  [60, 30],
])('first hour %i costs %i', (minutes, expected) => {
  expect(calcParkingFee(minutes)).toBe(expected);
});
```

Лістинг 3 src/insurance.ts

```
export const YOUNG_VEHICLE_BASE_UAH = 3000;
export const YOUNG_VEHICLE_MAX_AGE = 3;
export const MIDDLE_VEHICLE_MAX_AGE = 10;
export const MIDDLE_VEHICLE_BASE_UAH = 4000;
export const OLD_VEHICLE_BASE_UAH = 5000;
export const EXPERIENCED_DRIVER_FROM_YEARS = 3;
```

```
export const NOVICE_SURCHARGE_RATE = 0.25;
export const PERCENT_SCALE = 100;
export const MAX_ACCIDENT_FREE_DISCOUNT_PERCENT = 30;
export const KOPECKS_PER_UAH = 100;

export interface InsuranceInput {
  vehicleAge: number;
  driverExperience: number;
  accidentFreeDiscountPercent?: number;
}

function assertValidInput({
  vehicleAge,
  driverExperience,
  accidentFreeDiscountPercent = 0,
}: InsuranceInput): void {
  if (!Number.isSafeInteger(vehicleAge) || vehicleAge < 0) {
    throw new RangeError('INVALID_VEHICLE_AGE');
  }
  if (!Number.isFinite(driverExperience) || driverExperience < 0) {
    throw new RangeError('INVALID_DRIVER_EXPERIENCE');
  }
  if (
    !Number.isFinite(accidentFreeDiscountPercent) ||
    accidentFreeDiscountPercent < 0
  ) {
    throw new RangeError('INVALID_ACCIDENT_FREE_DISCOUNT');
  }
}
```

```
function calcBasePremium(vehicleAge: number): number {
  if (vehicleAge <= YOUNG_VEHICLE_MAX_AGE) return
YOUNG_VEHICLE_BASE_UAH;
  if (vehicleAge <= MIDDLE_VEHICLE_MAX_AGE) return
MIDDLE_VEHICLE_BASE_UAH;
  return OLD_VEHICLE_BASE_UAH;
}

function applyExperienceSurcharge(
  base: number,
  driverExperience: number,
): number {
  return driverExperience < EXPERIENCED_DRIVER_FROM_YEARS
    ? base * (1 + NOVICE_SURCHARGE_RATE)
    : base;
}

function applyAccidentFreeDiscount(
  amount: number,
  discountPercent: number,
): number {
  const effectiveDiscount = Math.min(
    discountPercent,
    MAX_ACCIDENT_FREE_DISCOUNT_PERCENT,
  );
  const [coefficient, exponent = '0'] = effectiveDiscount.toString().split('e');
  const [whole, fraction = ''] = coefficient.split('.');
  const scale = 10n ** BigInt(fraction.length - Number(exponent));
  const discount = BigInt(whole + fraction);
  const percentScale = BigInt(PERCENT_SCALE);
```

```
  const numerator =
    BigInt(amount) *
    BigInt(KOPECKS_PER_UAH) *
    (percentScale * scale - discount);
  const denominator = percentScale * scale;
  return roundPremium(numerator, denominator);
}

function roundPremium(numerator: bigint, denominator: bigint): number {
  // Positive half-up; preserve the public discount's decimal representation.
  const cents = (2n * numerator + denominator) / (2n * denominator);
  return Number(cents) / KOPECKS_PER_UAH;
}

/** Educational pricing assumptions are documented in report/lab3/PLAN.md. */
export function calcInsuranceCost(input: InsuranceInput): number {
  assertValidInput(input);
  const surcharged = applyExperienceSurcharge(
    calcBasePremium(input.vehicleAge),
    input.driverExperience,
  );
  return applyAccidentFreeDiscount(
    surcharged,
    input.accidentFreeDiscountPercent ?? 0,
  );
}
```

Лістинг 4 tests/insurance.test.ts

```
import { expect, it } from 'vitest';
import { calcInsuranceCost } from '../src/insurance';
```

```
it.each([
  [0.0074999, 2999.78],
  [0.0075, 2999.78],
  [0.0075001, 2999.77],
  [0.0345, 2998.97],
  [Number.MIN_VALUE, 3000],
])(
  'uses decimal half-up precision at discount %s',
  (accidentFreeDiscountPercent, expected) => {
    // 0.0075% of 3000 is 0.225; 2999.775 rounds to 2999.78.
    expect(
      calcInsuranceCost({
        vehicleAge: 0,
        driverExperience: 3,
        accidentFreeDiscountPercent,
      }),
    ).toBe(expected);
  },
);

it('accepts a small positive driving experience', () => {
  expect(calcInsuranceCost({ vehicleAge: 0, driverExperience: 0.01 })).toBe(
    3750,
  );
});

it.each([
  { discount: 12.34549, experience: 3, expected: 2629.64 },
  { discount: 12.3455, experience: 3, expected: 2629.64 },
```

```
  { discount: 12.34551, experience: 3, expected: 2629.63 },
  { discount: 12.3456, experience: 2, expected: 3287.04 },
  { discount: 0.0000001, experience: 3, expected: 3000 },
])(
  'rounds only final premium $discount $experience to $expected',
  ({ discount, experience, expected }) => {
    const result = calcInsuranceCost({
      vehicleAge: 0,
      driverExperience: experience,
      accidentFreeDiscountPercent: discount,
    });
    expect(result).toBe(expected);
    expect(result.toFixed(2)).toBe(expected.toFixed(2));
  },
);

it.each([
  { vehicleAge: 0, driverExperience: 3, expected: 2100 },
  { vehicleAge: 0, driverExperience: 2, expected: 2625 },
  { vehicleAge: 4, driverExperience: 3, expected: 2800 },
  { vehicleAge: 4, driverExperience: 2, expected: 3500 },
  { vehicleAge: 11, driverExperience: 3, expected: 3500 },
  { vehicleAge: 11, driverExperience: 2, expected: 4375 },
])(
  'combines category and experience $vehicleAge $driverExperience',
  ({ expected, ...input }) => {
    expect(
      calcInsuranceCost({ ...input, accidentFreeDiscountPercent: 30 }),
    ).toBe(expected);
  },
```

```
);

it('throws a meaningful RangeError for invalid age', () => {
  expect(() =>
    calcInsuranceCost({ vehicleAge: -1, driverExperience: 3 }),
  ).toThrow(RangeError);
});

it.each([
  [29, 2130],
  [29.99, 2100.3],
  [30, 2100],
  [30.01, 2100],
  [31, 2100],
  [100, 2100],
  [Number.MAX_VALUE, 2100],
])(
  'discount cap boundary %s percent costs %s',
  (accidentFreeDiscountPercent, expected) => {
    expect(
      calcInsuranceCost({
        vehicleAge: 0,
        driverExperience: 3,
        accidentFreeDiscountPercent,
      }),
    ).toBeCloseTo(expected, 8);
  },
);

it('caps the discount for an old car driven by a novice', () => {
```

```
  // 5000 + 1250 - 1875 = 4375, discount is on 6250, not on 5000.
  expect(
    calcInsuranceCost({
      vehicleAge: 11,
      driverExperience: 2,
      accidentFreeDiscountPercent: 31,
    }),
  ).toBe(4375);
});

it.each([
  [0, 3000],
  [0.01, 2999.7],
  [1, 2970],
  [10, 2700],
  [20, 2400],
])(
  'accident-free discount %s percent costs %s',
  (accidentFreeDiscountPercent, expected) => {
    expect(
      calcInsuranceCost({
        vehicleAge: 0,
        driverExperience: 3,
        accidentFreeDiscountPercent,
      }),
    ).toBe(expected);
  },
);

it('discounts the surcharged amount for an inexperienced driver', () => {
```

```
  // Independently: 3000 + 750 surcharge - 750 discount = 3000 UAH.
  expect(
    calcInsuranceCost({
      vehicleAge: 0,
      driverExperience: 2,
      accidentFreeDiscountPercent: 20,
    }),
  ).toBe(3000);
});

it.each([
  [0, 3750],
  [2, 3750],
  [2.99, 3750],
  [3, 3000],
  [3.01, 3000],
  [4, 3000],
])('experience boundary %s costs %i', (driverExperience, expected) => {
  expect(calcInsuranceCost({ vehicleAge: 0, driverExperience })).toBe(expected);
});

it.each([
  [4, 5000],
  [11, 6250],
])('novice with vehicle age %i costs %i', (vehicleAge, expected) => {
  expect(calcInsuranceCost({ vehicleAge, driverExperience: 2 })).toBe(expected);
});

it.each([
  [2, 3000],
```

```
  [3, 3000],
  [4, 4000],
  [9, 4000],
  [10, 4000],
  [11, 5000],
  [Number.MAX_SAFE_INTEGER - 1, 5000],
  [Number.MAX_SAFE_INTEGER, 5000],
])('age category boundary %i costs %i', (vehicleAge, expected) => {
  expect(calcInsuranceCost({ vehicleAge, driverExperience: 3 })).toBe(expected);
});

it.each([0, 1, 2, 3])('base cost for a young vehicle aged %i', (vehicleAge) => {
  expect(calcInsuranceCost({ vehicleAge, driverExperience: 3 })).toBe(3000);
});

it.each([
  -1,
  -0.01,
  0.5,
  NaN,
  Infinity,
  -Infinity,
  Number.MAX_SAFE_INTEGER + 1,
])('rejects invalid vehicle age %s', (vehicleAge) => {
  expect(() => calcInsuranceCost({ vehicleAge, driverExperience: 3 })).toThrow(
    'INVALID_VEHICLE_AGE',
  );
});

it.each([-1, -0.01, NaN, Infinity, -Infinity])(
```

```
  'rejects invalid driving experience %s',
  (driverExperience) => {
    expect(() =>
      calcInsuranceCost({ vehicleAge: 0, driverExperience }),
    ).toThrow('INVALID_DRIVER_EXPERIENCE');
  },
);

it.each([-1, -0.01, NaN, Infinity, -Infinity])(
  'rejects invalid accident-free discount %s',
  (accidentFreeDiscountPercent) => {
    expect(() =>
      calcInsuranceCost({
        vehicleAge: 0,
        driverExperience: 3,
        accidentFreeDiscountPercent,
      }),
    ).toThrow('INVALID_ACCIDENT_FREE_DISCOUNT');
  },
);
```

5 Виконаний рефакторинг

Зафіксовано 16 окремих Refactor-комітів, по одному в кожному циклі. Деякі кроки невеликі, наприклад іменування тарифу; змістовні структурні кроки наведено нижче. Жоден із 16 повних прогонів після рефакторингу не завершився невдачею.

Таблиця 6 – Структурні зміни під захистом тестів

Коміт	Було	Стало та навіщо
3dd6f93	Перевірка тривалості в основній функції	assertValidDuration відокремлює контракт від обчислення.
0692c42	Порогові тарифи разом із валідацією	calcBaseFee окремо повертає обмежену базову ціну.
9dcb104	Нічна знижка в основному калькуляторі	applyRates відокремлює знижки від бази.
00fc60b	Літерали 3, 10, 4000, 5000 і розгалуження в калькуляторі	Іменовані константи та calcBasePremium локалізують політику віку.
f6587b0	Умова стажу та множник 1,25 у калькуляторі	applyExperienceSurcharge із константами межі й надбавки.
dcd4409	Округлення всередині знижки плюс повторне через Number	roundPremium округлює раціональну суму; повторний двійковий крок видалено.

Знімок коду з Green третього циклу показує валідацію, після якої основна функція самостійно перевіряла вікові межі. Цей вміст збережено до зміни структури.

Insurance cycle 03 refactor-before

Execution-log viewer — original captured process output, not a terminal emulator.

Exact source snapshot: insurance-03/green.source.txt

```
export const YOUNG_VEHICLE_BASE_UAH = 3000;

export interface InsuranceInput {
  vehicleAge: number;
  driverExperience: number;
  accidentFreeDiscountPercent?: number;
}

function assertValidInput({
  vehicleAge,
  driverExperience,
  accidentFreeDiscountPercent = 0,
}: InsuranceInput): void {
  if (!Number.isSafeInteger(vehicleAge) || vehicleAge < 0) {
    throw new RangeError('INVALID_VEHICLE_AGE');
  }
  if (!Number.isFinite(driverExperience) || driverExperience < 0) {
    throw new RangeError('INVALID_DRIVER_EXPERIENCE');
  }
  if (
    !Number.isFinite(accidentFreeDiscountPercent) ||
    accidentFreeDiscountPercent < 0
  ) {
    throw new RangeError('INVALID_ACCIDENT_FREE_DISCOUNT');
  }
}

export function calcInsuranceCost(input: InsuranceInput): number {
  assertValidInput(input);
  if (input.vehicleAge <= 3) return YOUNG_VEHICLE_BASE_UAH;
  if (input.vehicleAge <= 10) return 4000;
  return 5000;
}
```

Рисунок 5 – Страховий модуль до виділення базової ціни

Після рефакторингу calcBasePremium містить ті самі розгалуження, а основна функція викликає його. Усі межі та суми мають назви, тому зміна політики не вимагає пошуку магічних чисел у потоці обчислення.

Insurance cycle 03 refactor-after

Execution-log viewer — original captured process output, not a terminal emulator.

Exact source snapshot: insurance-03/refactor.source.txt

```
export const YOUNG_VEHICLE_BASE_UAH = 3000;
export const YOUNG_VEHICLE_MAX_AGE = 3;
export const MIDDLE_VEHICLE_MAX_AGE = 10;
export const MIDDLE_VEHICLE_BASE_UAH = 4000;
export const OLD_VEHICLE_BASE_UAH = 5000;

export interface InsuranceInput {
  vehicleAge: number;
  driverExperience: number;
  accidentFreeDiscountPercent?: number;
}

function assertValidInput({
  vehicleAge,
  driverExperience,
  accidentFreeDiscountPercent = 0,
}: InsuranceInput): void {
  if (!Number.isSafeInteger(vehicleAge) || vehicleAge < 0) {
    throw new RangeError('INVALID_VEHICLE_AGE');
  }
  if (!Number.isFinite(driverExperience) || driverExperience < 0) {
    throw new RangeError('INVALID_DRIVER_EXPERIENCE');
  }
  if (
    !Number.isFinite(accidentFreeDiscountPercent) ||
    accidentFreeDiscountPercent < 0
  ) {
    throw new RangeError('INVALID_ACCIDENT_FREE_DISCOUNT');
  }
}

function calcBasePremium(vehicleAge: number): number {
  if (vehicleAge <= YOUNG_VEHICLE_MAX_AGE) return YOUNG_VEHICLE_BASE_UAH;
  if (vehicleAge <= MIDDLE_VEHICLE_MAX_AGE) return MIDDLE_VEHICLE_BASE_UAH;
  return OLD_VEHICLE_BASE_UAH;
}

export function calcInsuranceCost(input: InsuranceInput): number {
  assertValidInput(input);
  return calcBasePremium(input.vehicleAge);
}
```

Рисунок 6 – Той самий модуль після рефакторингу

6 Перевірки та фактичне покриття

Таблиця 7 – Підсумкові перевірки

Команда	Фактичний результат
npm test -- tests/parking.test.ts	40 тестів пройшли
npm test -- tests/insurance.test.ts	69 тестів пройшли

Команда	Фактичний результат
npm test	254 тести, 6 файлів; невдалих 0
Окремий прогін лабораторних №1 і №2	145 тестів, 4 файли; невдалих 0
npm run lint	exit 0
npm run typecheck	exit 0
npm run format:check	exit 0; усі файли відповідають Prettier
npm run coverage	exit 0; попередні пороги й окремий поріг страхування пройдено

Окремий фінальний прогін паркування підтверджує 40 успішних сценаріїв, включно з межами тривалості, нічним і вихідним тарифами.

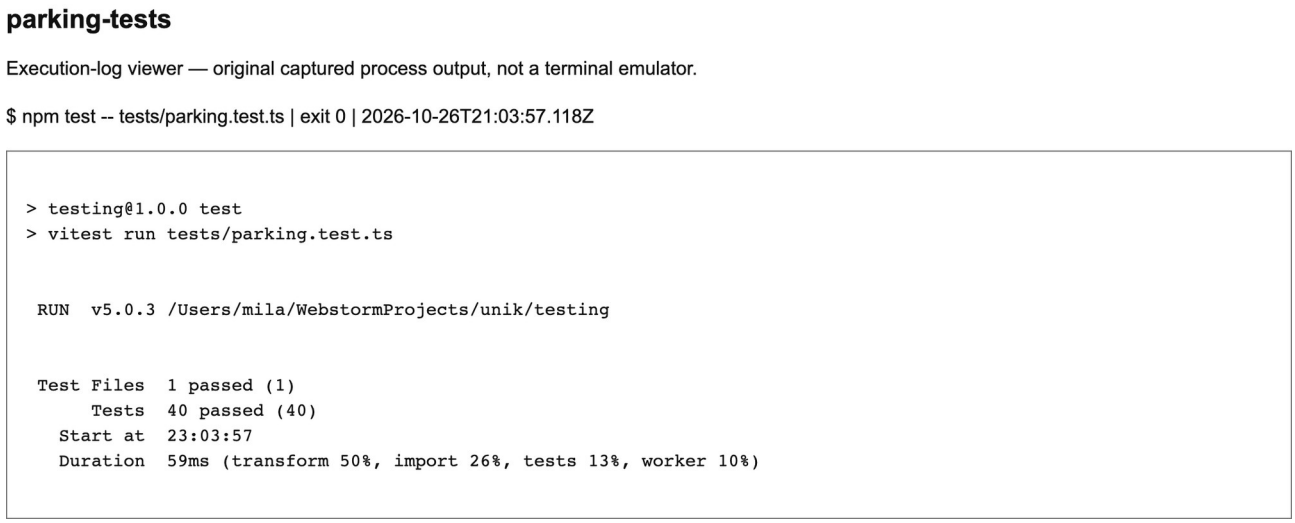


Рисунок 7 – Успішні тести показового модуля

Окремий фінальний прогін страхування підтверджує 69 успішних тестів: позитивні, негативні, параметризовані, межові та комбіновані сценарії.

insurance-tests

Execution-log viewer — original captured process output, not a terminal emulator.

$ npm test -- tests/insurance.test.ts | exit 0 | 2026-10-26T21:17:51.407Z

```
> testing@1.0.0 test
> vitest run tests/insurance.test.ts


 RUN  v5.0.3 /Users/mila/WebstormProjects/unik/testing


 Test Files  1 passed (1)
      Tests  69 passed (69)
   Start at  23:17:51
   Duration  62ms (transform 53%, import 24%, tests 14%, worker 9%)
```

Рисунок 8 – Успішні тести варіанта №12

Повний прогін містить обидва нові модулі й попередні лабораторні. Усі 254 тести в шести файлах успішні.

full-tests

Execution-log viewer — original captured process output, not a terminal emulator.

$ npm test | exit 0 | 2026-10-26T21:17:51.696Z

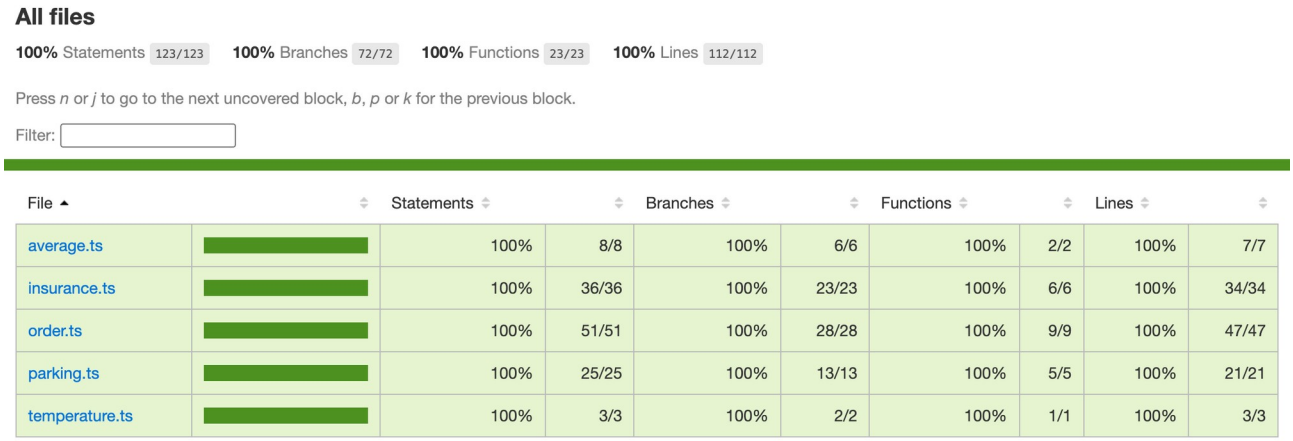
```
> testing@1.0.0 test
> vitest run


 RUN  v5.0.3 /Users/mila/WebstormProjects/unik/testing


 Test Files  6 passed (6)
      Tests  254 passed (254)
   Start at  23:17:51
   Duration  86ms (transform 57%, import 22%, worker 11%, tests 10%)
```

Рисунок 9 – Повний регресійний набір

Playwright відкрив справжній HTML-звіт coverage/index.html. Таблиця показує Statements, Branches, Functions і Lines для всіх п’яти модулів; це не відтворена вручну таблиця.



All files

100% Statements 123/123 **100%** Branches 72/72 **100%** Functions 23/23 **100%** Lines 112/112

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
average.ts	100%	8/8	100%	6/6	100%	2/2	100%	7/7
insurance.ts	100%	36/36	100%	23/23	100%	6/6	100%	34/34
order.ts	100%	51/51	100%	28/28	100%	9/9	100%	47/47
parking.ts	100%	25/25	100%	13/13	100%	5/5	100%	21/21
temperature.ts	100%	3/3	100%	2/2	100%	1/1	100%	3/3

Рисунок 10 – Загальна таблиця фактичного V8-покриття

Детальна сторінка coverage/insurance.ts.html показує 100% і 23/23 гілки страхування. Повний знімок сторінки зі всім кодом збережено окремо в insurance-branch-coverage.png.



All files **insurance.ts**

100% Statements 36/36 **100%** Branches 23/23 **100%** Functions 6/6 **100%** Lines 34/34

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1  1x  export const YOUNG_VEHICLE_BASE_UAH = 3000;
2  1x  export const YOUNG_VEHICLE_MAX_AGE = 3;
3  1x  export const MIDDLE_VEHICLE_MAX_AGE = 10;
4  1x  export const MIDDLE_VEHICLE_BASE_UAH = 4000;
5  1x  export const OLD_VEHICLE_BASE_UAH = 5000;
6  1x  export const EXPERIENCED_DRIVER_FROM_YEARS = 3;
7  1x  export const NOVICE_SURCHARGE_RATE = 0.25;
8  1x  export const PERCENT_SCALE = 100;
9  1x  export const MAX_ACCIDENT_FREE_DISCOUNT_PERCENT = 30;
10 1x  export const KOPECKS_PER_UAH = 100;
11
12     export interface InsuranceInput {
13       vehicleAge: number;
14       driverExperience: number;
15       accidentFreeDiscountPercent?: number;
16     }
17
18     function assertValidInput({
19       vehicleAge,
20       driverExperience,
21       accidentFreeDiscountPercent = 0,
22     }: InsuranceInput): void {
23 69x    if (!Number.isSafeInteger(vehicleAge) || vehicleAge < 0) {
24  8x      throw new RangeError('INVALID_VEHICLE_AGE');
25        }
26 61x    if (!Number.isFinite(driverExperience) || driverExperience < 0) {
27  5x      throw new RangeError('INVALID_DRIVER_EXPERIENCE');
28        }
29 56x    if (
30          !Number.isFinite(accidentFreeDiscountPercent) ||
31          accidentFreeDiscountPercent < 0
32        ) {
33  5x      throw new RangeError('INVALID_ACCIDENT_FREE_DISCOUNT');
34        }
35     }
36
37     function calcBasePremium(vehicleAge: number): number {
38 51x    if (vehicleAge <= YOUNG_VEHICLE_MAX_AGE) return YOUNG_VEHICLE_BASE_UAH;
39 13x    if (vehicleAge <= MIDDLE_VEHICLE_MAX_AGE) return MIDDLE_VEHICLE_BASE_UAH;
```

Рисунок 11 – Детальна сторінка покриття страхового модуля

Таблиця 8 – Чисельники і знаменники покриття

Обсяг	Statements	Branches	Functions	Lines
Паркування	25/25	13/13	5/5	21/21
Страхування	36/36	23/23	6/6	34/34
Увесь проєкт	123/123	72/72	23/23	112/112

Для кожного показника таблиці отримано 100%. Vitest перевіряє branches ≥90% для src/insurance.ts, а scripts/check-insurance-coverage.mjs незалежно перевіряє лічильники 23/23 у JSON-звіті. Збережено попередні глобальні пороги: Lines ≥80%, Branches ≥80%, Functions 100%, а також окрему перевірку calcInstallment ≥90%.

Перевірка ESLint завершилася з exit 0 без діагностик. Команду та оригінальний вивід збережено поряд зі знімком.

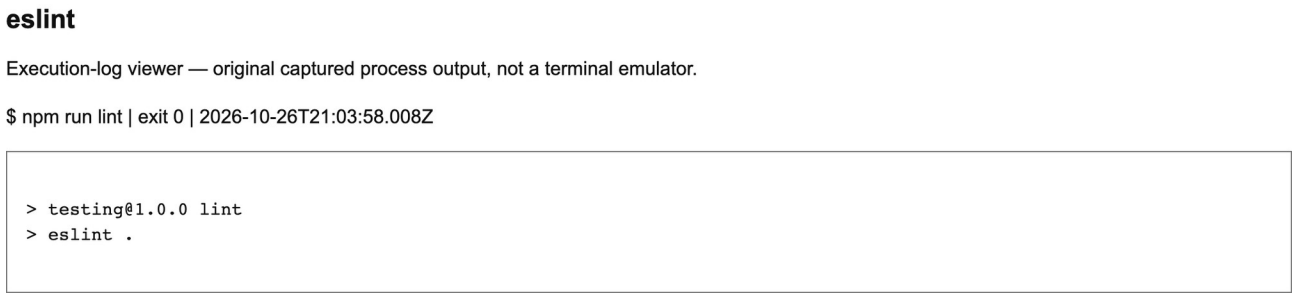


Рисунок 12 – Успішна перевірка ESLint

TypeScript виконав tsc --noEmit без помилок. Публічні сигнатури й тести перевірено за чинним strict-конфігом репозиторію.

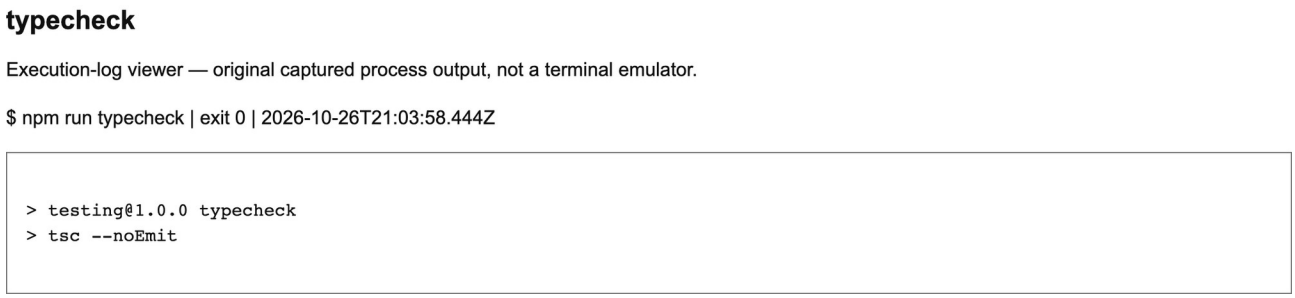


Рисунок 13 – Успішна перевірка TypeScript

Перевірка Prettier підтвердила форматування всіх охоплених файлів. Форматувальник і його конфігурацію з попередніх робіт збережено.

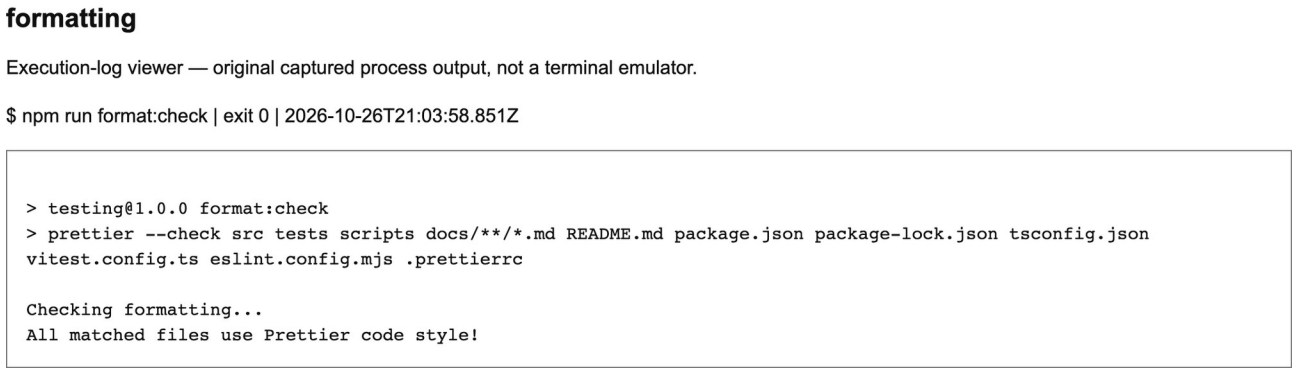


Рисунок 14 – Успішна перевірка форматування

Усі PNG створено автоматично через Playwright та Chrome. Для процесів використано чітко позначений execution-log viewer; DOM-текст перевірено на точний збіг із stdout, після якого йде stderr. Оригінальні потоки збережено окремо, разом із командою, часом, кодом завершення та знімками коду. Жодного результату тестування або інтерфейсу термінала не домальовано.

7 Аналіз результату за кроком 9

7.1 Кількість рядків та непокритий код

Новий продуктовий код містить 135 фізичних рядків: parking.ts – 45, insurance.ts – 90. Без порожніх рядків – 122: відповідно 39 і 83. V8 враховує 55 виконуваних рядків: 21 і 34. Усі вони покриті, непокритих виконуваних рядків – 0. Фізичні рядки включають типи, константи, дужки та коментарі, тому їх не слід ототожнювати зі знаменником Lines у V8.

7.2 Покриття гілок і додаткові тести

Страхування має 23/23 покриті гілки, тобто 100% за порога 90%; паркування – 13/13; весь проєкт – 72/72. Уже перший фінальний вимір після циклів показав 100%, тому додаткові тести лише для досягнення порога не знадобилися. Восьмий страховий цикл додано через реальну похибку округлення, а не для збільшення метрики. Покриття підтверджує виконання гілок, але правильність чисел забезпечують незалежні очікування.

7.3 Кроки Refactor і стабільність тестів

Виконано 16 кроків Refactor: 8 у паркуванні та 8 у страхуванні. У кожному після змін запускався повний наявний набір, і всі 16 запусків завершилися з exit 0. Під час цих рефакторингів тести не ламалися. У Red очікувано падали перевірки ще відсутньої поведінки; це не невдачі рефакторингу.

7.4 Спокуса реалізувати майбутні вимоги

Місцями для передчасного узагальнення були перша базова ціна та застосування знижки. Після вимоги 3000 грн було легко одразу додати всі вікові категорії, а після першої знижки – одразу межу 30%. Фактичні Green-коміти цього не роблять: 0365113 повертає лише початкову ціну, а 2efe14c ще не обмежує відсоток. Майбутню поведінку додано тільки після наступного Red. Це зберегло перевірювану причину кожної зміни.

7.5 Вплив TDD на дизайн страхування

Окремі тести валідації привели до assertValidInput; тести вікових меж – до calcBasePremium; взаємодія стажу й знижки – до послідовного застосування правил. Регресія 0,0075% показала, що фінансовий контракт не можна покладати лише на двійкове множення. Через це точне округлення винесено в окрему функцію на bigint. Калькулятор залишився чистою функцією без залежності від UI, часу чи мережі.

7.6 Відповіді на контрольні питання

1. Red – написати перевірку нової вимоги й побачити очікуваний збій; Green – додати мінімальний код до успіху; Refactor – поліпшити структуру без зміни поведінки й повторно перевірити тести.

2. Невдалий запуск доводить, що тест виявляє відсутню поведінку. Новий тест може одразу пройти, якщо поведінка вже реалізована або тест не перевіряє потрібний сценарій, наприклад не має дієвого твердження.

3. Мінімальний Green не вводить неперевірених правил. Наприклад, у 0365113 для страхування лише повертається 3000 грн; інші вікові ціни додано після наступного Red.

4. Зелений набір дає перевірену поведінкову основу. Якщо до рефакторингу тести вже падають, важко відрізнити стару помилку від нової, спричиненої структурною зміною.

5. У четвертому показовому циклі іменовано тривалість години й додатковий тариф; далі відокремлено валідацію, базовий розрахунок, ставки та округлення. У цій реалізації ці кроки мають окремі коміти.

6. Для 600 хв необмежена плата 210 грн. Правильно: обмежити до 200, потім нічний тариф дає 100. Якщо поділити 210 навпіл до обмеження, залишиться 105. Тому порядок обмеження і знижки суттєвий.

7. Зворотний зв’язок про архітектуру виникає, коли тестування окремих правил показує різні відповідальності. Тут вікові межі, стаж і точність копійок привели до невеликих незалежних функцій.

8. Один фінальний коміт не доводить, що перевірки передували коду та справді падали. Методичка оцінює процес; окремі коміти з журналами роблять його перевірюваним.

9. Класичний TDD менш придатний для дослідницьких прототипів із невідомим контрактом, суб’єктивної оцінки дизайну та експериментальних алгоритмів без готового оракула. Спочатку потрібні дослідження або інший спосіб оцінки.

10. Найкращий промисловий сценарій – нове чітке бізнесове правило або відтворена регресія: відомі вхід і незалежне очікування дозволяють швидко отримати Red, мінімальний Green і безпечний Refactor.

8 Рецензія на роботу однокурсника

Статус: ОЧІКУЄ ВИКОНАННЯ. Посилання на репозиторій або Pull Request однокурсника не надано, і жодну доступну роботу не ідентифіковано як роботу конкретного однокурсника. Тому завершеної рецензії немає. Власний

аудит історії не підміняє взаємне рецензування. Нижче наведено шаблон для заповнення після отримання реального посилання.

Таблиця 9 – Шаблон майбутньої взаємної рецензії

Критерій	Що перевірити та зафіксувати
Автор і джерело	ПІБ однокурсника, URL, гілка, діапазон хешів; ще не заповнено.
Порядок TDD	Відповідність Red → Green → Refactor; для кожної поведінки порівняти появу тесту та реалізації. Навести реальні хеші й приклад.
Мінімальність Green	Переглянути diff кожного Green; чи додано тільки поточну вимогу, чи майбутню поведінку. Навести конкретний фрагмент.
Рефакторинг	Вказати структурні зміни без нової поведінки, їхню користь і фактичний стан тестів після змін.
Підсумок	Сформувати обґрунтований висновок після перегляду історії; зараз висновок не сформовано.

9 Репозиторій і відтворення результатів

Наявний GitHub-репозиторій: https://github.com/milunchik/testing
Pull Request лабораторної №3: https://github.com/milunchik/testing/pull/3

Гілку опубліковано з підтвердженою HTTPS-авторизацією. Pull Request створено як draft до feature/lab2-vitest-installment, щоб ізолювати зміни лабораторної №3. Злиття не виконувалося. Після додавання звіту гілка містить також документувальні коміти, які не змінюють послідовність TDD.

Для перевірки: npm test; npm run lint; npm run typecheck; npm run format:check; npm run coverage. Автоматичний аудит: node scripts/lab3-audit.mjs. Повторне отримання PNG: npm run evidence:lab3. Журнали та метадані

зберігаються у report/lab3/evidence, знімки – у report/lab3/screenshots. Архів справжнього HTML-покриття – report/lab3/coverage-html.zip.

Скріншот історії фіксує момент після завершення TDD; наступні службові та документувальні коміти доповнюють його. Для остаточного стану слід виконати git log --oneline і git status. Хеші всіх 48 етапів у таблиці залишаються чинними; переписування історії не виконувалося.

10 Висновки щодо складнощів і застосування TDD

Реалізовано обидва модулі у послідовних циклах із реальними невдалими перевірками. Найскладнішими виявилися відокремлення числових припущень від вимог методички, дотримання мінімальності Green та точність округлення. Окремий восьмий цикл страхування продемонстрував практичну користь Red: він виявив заниження на копійку, хоча попередні приклади й перевірки вже проходили.

TDD допоміг сформувати невеликі функції з окремими відповідальностями та дозволив безпечно змінити структуру. Для чистих бізнесових калькуляторів підхід доречний: вимоги мають чіткі вхідні дані, межі та незалежні очікування. У промисловій роботі найкраще застосовувати його до усталених бізнесових правил і регресій, доповнюючи інтеграційними та системними перевірками.

Для пошукового прототипування, суб’єктивного візуального дизайну та експериментальних алгоритмів із невідомим очікуваним результатом класичний TDD потребує попереднього дослідження. Натомість для відтвореної помилки або конкретного правила він дає швидкий зворотний зв’язок. Показник 100% не замінює правильного контракту, тому звіт містить і межові приклади, і пояснення припущень, і справжню історію тестів.

Залишаються навчальні дії: отримати роботу однокурсника й виконати реальну взаємну рецензію та захистити роботу. Ці кроки не позначено як виконані.